CS 645

Security and Privacy in Computer Systems

Lecture 2

- Authentication systems

Course information

- When and where?

  Monday 6:00 – 8:50pm @ Jersey city CS 645 (1j1)

- Course webpage (general information, syllabus, course slides, projects, etc.):

  https://canvas.njit.edu/

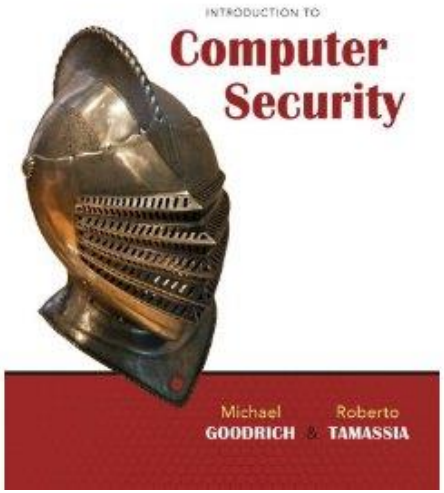
- Class attendance is strongly recommended
- If you miss a lecture, it is your responsibility to find out what happened in class

Course information

- Professor contact info:
  - Email: fuad.hamidli@njit.edu
  - Office hours @ NJIT JC:
    - Tuesday 4:00 – 5:00pm
    - Wednesday 5:00-6:00 pm
    - Also, by appointment via email

Textbook

- *"Introduction to Computer Security"*
  by M. Goodrich and R. Tamassia,
  Addison Wesley, 2010, ISBN: 032151294



- In addition, course material will include research articles from electronic databases such as:
  - ACM Digital Library: http://dl.acm.org
  - IEEE Xplore: http://ieeexplore.ieee.org
  - Science Direct: http://www.sciencedirect.com

Grading policy

- 3 mini-projects 45%
- Midterm Exam 25%
- Final Exam 30%

Up to 10% extra credit for active participation in class

No make up exams

Mini-projects

- Mini-projects
  - must be **typed**
  - are due in the beginning of the class
    - If you cannot attend class, assignment should be sent by email 1 hour before class starts

Academic integrity

- The NJIT University code on academic integrity will be followed:
  https://www.njit.edu/policies/sites/policies/files/academic-integrity-code.pdf

(Some excerpts): You should not:

- Take an examination for another student.
- Plagiarize, in part, written, oral or graphic work which was authored or prepared by another.
- Fail to acknowledge that the work submitted for credit is the work of a collaboration.
- Cheat on an examination.

Note in particular that cheating on exams, copying homework assignments and exam papers, and plagiarizing (in full or in part) someone else's work is forbidden. Students will be sent to the Dean of Students when such situations arise, for disciplinary actions.

Academic Integrity

- *Please note that it is my professional obligation and responsibility to report any academic misconduct to the Dean of Students Office. Any student found in violation of the code by cheating, plagiarizing or using any online software inappropriately will result in disciplinary action. This may include a failing grade of F, and/or suspension or dismissal from the university.*

- *As part of projects, students must turn in code or work that has fully been written by him/her and no-one else. Any submitted text or code (even few lines) obtained through the Internet or otherwise, or is product of someone else's work, risks severe punishment, as outlined by the University; all parties of such interaction receive automatically 0 and grade is lowered by one or two levels.*

Important dates

- Last day to withdraw: Apr 1
- Reading days: May 1, May 2
- Final exam: May 8 (to be confirmed)

NJIT Spring 2024 academic calendar:

https://www.njit.edu/registrar/spring-2024-academic-calendar

Important dates

2/14/24	Project 1 posted
2/28/24	Project 1 due
3/6/24	Midterm exam
3/27/24	Project 2 posted
4/10/24	Project 2 due
4/10/24	Project 3 posted
4/24/24	Project 3 due
5/8/24	Final exam (to be confirmed)

(project dates are tentative and might change)

Other important stuff

- You are advised to take notes

- Prerequisites
  - There are no specific course prerequisites for the course, but students are expected to enter this course with a basic knowledge of operating systems, networking, algorithms, and data structures.
  - Also, students should be able to program in Java and C/C++.

Other important stuff

- When you send me an email about this class, please start the subject line with “CS.645”

Course outline

- Introduction (overview of course topics, security goals, overview of attacks)
- Crypto crash course
- Access control mechanisms
- Operating systems security
- Software security, Secure Programming
- Malicious code, Malware, Rootkits
- Web security
- Trusted computing
- Introduction to security of networked systems
- Privacy and anonymity on the Web
- Content protection, Software obfuscation, Digital rights management
- Database security
- Security of electronic voting
- Computer crime - laws and ethics, Security & privacy policy (Sarbanes Oxley, HIPAA)
- Miscellaneous topics: side-channel attacks, gaming security, information assurance (common criteria), risk analysis

Last Week

- Introduction
  - Security goals (confidentiality, integrity, authentication, non-repudiation)
  - Threats and attacks
- Models for access control
- Cryptographic concepts
  - Hash functions
  - Symmetric key encryption
  - Message authentication codes (MACs)
  - Public key signature
  - Public key encryption

Authentication systems

Identification (entity authentication)

- Used to facilitate access control to a resource
- Entity authentication is the process by which one party is assured:
  - of the identity of a second party involved in a protocol, and
  - the second party has actually participated (i.e., is active immediately prior to the the time evidence is acquired)

- We'll use the terms authentication and identification alternatively

Objectives of an identification protocol:

- If A and B are honest, A is able to successfully authenticate itself to B
- *Transferability*: B cannot reuse an identification exchange with A to successfully impersonate A to a third party C
- *Impersonation*: C (different than A) cannot pretend to be A and convince B to complete and accept A's identity
  - The above remains true even if:
    - C observes a large number of previous interactions between A and B
    - C has participated in previous protocol executions with either or both A and B
    - Multiple instances of the protocol, possibly initiated by C, may be run simultaneously

Objectives of an identification protocol:

- User registration is required prior to an identification protocol

Basis of identification

- Something you know
  - password, Personal Identification Number (PIN), secret key
- Something you have
  - Passport, magnetic-strip card, smartcard, hardware tokens
- Something you are
  - Biometrics: handwritten signatures, fingerprints, voice, retinal patterns, keystroke dynamics, facial geometry
- These can be combined

(Text) Passwords

- A shared secret between a user and a system, which allows the user to authenticate itself to the system
- Identification is based on *userid* and *psswd*
- Simplest way: store password in the clear on the system
  - No protection against a superuser (root access)
  - Backups contain the password in the clear
- Better way: one-way password file
  - Password file stores a one-way function of each password (e.g., cryptographic hash)
    - E.g., /etc/shadow

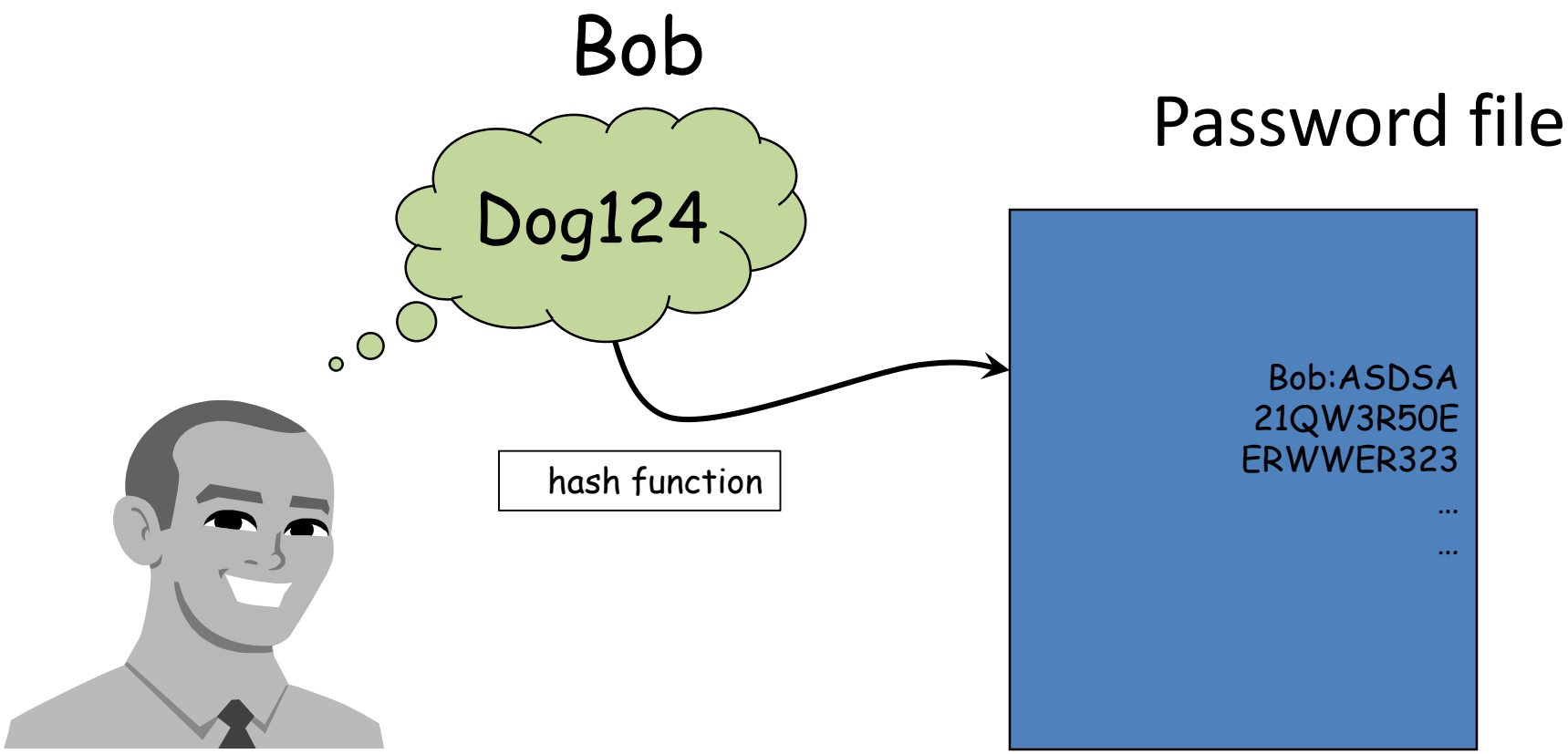
Passwords

- A short sequence of characters used as a means to authenticate someone via a secret that they know.

- Userid: _____
- Password: _____

- The system compares the userid, password information with what it has stored
- If the check succeeds, access is granted

How is a password stored?



Strong passwords

- What is a strong password
  - UPPER/lower case characters
  - Special characters
  - Numbers
- When is a password strong?
  - Seattle1
  - M1ke03
  - P@$$w0rd
  - TD2k5secV

Password complexity

A fixed 6 symbols password:

- Numbers
  10^6 = 1,000,000
- UPPER or lower case characters (case insensitive)
  26^6 = 308,915,776
- UPPER and lower case characters (case sensitive)
  52^6 = 19,770,609,664
- 32 special characters (&, %, $, £, “, |, ^, § , etc.)
  32^6 = 1,073,741,824

94 practical symbols available

- 94^6 = 689,869,781,056

ASCII standard 7 bit 2^7 =128 symbols

- 128^6 = 4,398,046,511,104

Password length

26 UPPER/lower case characters = 52 characters
10 numbers
32 special characters
=> 94 characters available

5 characters: 94^5 =	7,339,040,224
6 characters: 94^6 =	689,869,781,056
7 characters: 94^7 =	64,847,759,419,264
8 characters: 94^8 =	6,095,689,385,410,816
9 characters: 94^9 =	572,994,802,228,616,704

Longer passwords are better

Password validity: Brute force test

Password does not change for 60 days

how many passwords should I try for each second?

5 characters:	1,415 PW /sec
6 characters:	133,076 PW /sec
7 characters:	12,509,214 PW /sec
8 characters:	1,175,866,008 PW /sec
9 characters:	110,531,404,750 PW /sec

Strong passwords

A strong password includes characters from at least three of the following groups:

Group	Example
Lowercase letters	a, b, c, ...
Uppercase letters	A, B, C, ...
Numerals	0, 1, 2, 3, 4, 5, 6, 7, 8, 9
Non-alphanumeric (symbols)	( ) ` ~ ! @ # $ % ^ & * - + = | \ { } [ ] : ; " ' < > , . ? /
Unicode characters	€, Γ, ƒ, and λ

Use pass phrases eg. "I re@lly want to buy 11 Dogs!"

Strong passwords

Password policies are getting out of control

http://cacm.acm.org/blogs/blog-cacm/123889-password-policies-are-getting-out-of-control/fulltext

Attacks against password-based authentication

- Exhaustive search (brute force)
  - an attacker attempts to guess a user password by trying all possible strings
- Dictionary attack
  - Only try words in a dictionary (e.g., 150,000 words) instead of exhaustive search (e.g., 8-character lower case password takes 1.3 years)
  - Has a high probability of success

Attacks against password-based authentication

- Online attacks: the system is involved in each authentication attempt
  - Penalty for failed attempts (e.g., lock-out after 3 attempts)
- Off-line attacks: the system is not involved
  - Example: attacker has access to the password file

- Or so it should be…
  - Feb 2014: unlimited number of attempts on Amazon's mobile apps for iOS and Android
  - http://www.networkworld.com/article/2174753/byodmazon-com-security-slip-allowed-unlimited-pass/byod/amazon-com-security-slip-allowed-unlimited-password-guesses.html

Off-line attacks

- The basic approach to guessing passwords from the password file is to conduct a **dictionary attack**, where each word in a dictionary is hashed and the resulting value is compared with the hashed passwords stored in the password file.
- A dictionary of 500,000 “words” is often enough to discover most passwords.
- Dictionary attacks
  - Generally, not successful at finding a particular user’s password, but find many passwords given access to a password file

Off-line attacks

- *Pre-computed* dictionary attacks
  - The attacker pre-computes and stores the hashes of all the words in the dictionary
  - Trade-off between *time* and *space* by using a list of dictionary words to pre-compute and store a list of pre-computed passwords
  - Makes attack almost instantaneous (although pre-computation time may be considerable)
  - Particularly effective when a large number of passwords are to be cracked at once

Rainbow Tables

- What if the the pre-computed dictionary is too large to store? Use a rainbow table!
- Based on an earlier concept, called a Hellman table
- The concept of a hash chain using a hash function and a reduction function
  - Hash function H is a cryptographic hash function (maps a password to a hash)
    - E.g.: H(“273662”) = “222f00dc4b7f9131c89cff641d1a8c50“
  - Reduction function R maps a hash to a character string that looks like a password
    - E.g.: R(“222f00dc4b7f9131c89cff641d1a8c50“) = “222004”

Rainbow Tables

- Given a password p_1, construct a hash chain:

$$p_1 \rightarrow c_1=H(p_1) \rightarrow p_2=R(c_1) \rightarrow$$
$$c_2=H(p_2) \rightarrow p_3=R(c_2) \rightarrow$$
$$c_3=H(p_3) \rightarrow p_4=R(c_3) \rightarrow$$
$$\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots \rightarrow$$
$$c_{k-2}=H(p_{k-2}) \rightarrow p_{k-1}=R(c_{k-2}) \rightarrow$$
$$c_{k-1}=H(p_{k-1}) \rightarrow p_k=R(c_{k-1})$$

- The length of the hash chain is k
- We only store the first and last element in the hash chain (p_1 and p_k)

Rainbow Tables

- We create a table that stores multiple hash chains:

	Starting point (plaintext)	Ending point (plaintext) after applying k-1 times H and R
1	p^1_1	p^1_k
2	p^2_1	p^2_k
3	p^3_1	p^3_k
…	…	…

- How to use the table? Say we want to "crack" a value "c" (let's call this the "target hash")
  - This means that we want to find p such that: $h(p) = c$

Rainbow Tables – Lookup procedure

How to use the table? Say we want to "crack" a target hash c:

1) d = c

2) Repeat k times

- If not the first iteration, then d = H(q)
- q = R(d)
- If q appears in any of rows of the table's lastcolumn then
  - found = 1; i = row number where q appears
  - Break

3) If (found == 1), then the password associated with c is in the chain that corresponds to the i-th row of the table (with high probability)

4) Regenerate the i-th hash chain, starting from p^i_1:

$p_1 \rightarrow c_1=H(p_1) \rightarrow p_2=R(c_1) \rightarrow c_2=H(p_2) \rightarrow p_3=R(c_2) \rightarrow c_3=H(p_3) \rightarrow$
$p_4=R(c_3) \rightarrow \ldots \rightarrow p_k=R(c_{k-1})$

5) In step 4) check if the target hash c matches one of the hashes in this i-th chain. If so, then the password is the plaintext that precedes c in the chain.

Rainbow Tables

- What is the reduction in required storage?
- Each hash chain replaces k pairs of (password, hash) in the regular pre-computed cracking method
- So, we need k times less storage

Rainbow Tables

- If no plaintext matched with any of the values in the last column, then the password we're looking for does NOT exist in any of the hash chains stored in the table
- It is possible that step 3) returns a match, but step 5) doesn't return a match. This is a false positive (the reduction function resulted in a collision)
- To reduce the probability of a false positive, we can use multiple tables, each using a different reduction function R

Rainbow Tables

- Phillipe Oechslin introduced Rainbow Tables in a 2003 research article titled *"Making a Faster Cryptanalytic Time-Memory Trade-Off"*
- Use only one table, but use k different reduction functions R_1, R_2, …, R_k for each of the k steps in the construction of a chain
- This implies a slightly different lookup procedure (step 2):
  - First apply the last of the reduction functions R_k to obtain $q_1=R_k(c)$ and then check if p_1 appears in any of the rows of the last column
  - If not, apply the next reduction function $q_2 = R_{k-1}(H(q_1))$, etc.
  - Steps 3) and 5) need to also be modified accordingly

Rainbow Tables - Conclusion

- Rainbow tables are thus (also) used for pre-computed attacks
- How do they compare with a regular pre-computed dictionary attack?
  - Computation
  - Storage

Passwords: How to improve their security?

- Rules to prevent “weak” passwords
  - Minimum size
  - Include characters from several categories (numeric, uppercase, non-alphanumeric)
  - Give user suggestions/guidelines in choosing passwords
    - e.g., think of a sentence and select letters from it, “It’s 12 noon and I am hungry” => “I’S12&IAH”
- Frequent change, mandate password expiration

Passwords: How to improve their security?

- Slow down password verification
  - the hash function for password verification is made more computationally expensive
  - this can be done, e.g., by iterating the computation of the hash function multiple times
  - this change might affect legitimate users as well, but to a smaller degree
- Limit the number of unsuccessful password guesses
  - User account is locked after 3 unsuccessful attempts
- Use password salting

Passwords: How to improve their security?

- Prevent direct access to password file
  - Password file is not accessible to ordinary users

- It is always a challenge to find a good balance between password memorability and resistance to dictionary attacks
  - Unpredictability and usability of passwords is hard to achieve at the same time

Password salting

- Mechanism:
  - Password is pre-pended with a randomly chosen value called “salt” before applying the one-way function: h(salt, psswd)
  - Password file stores both hashed password and salt
  - On authentication, the system appends the salt to the password before hashing

How Password Salt Works

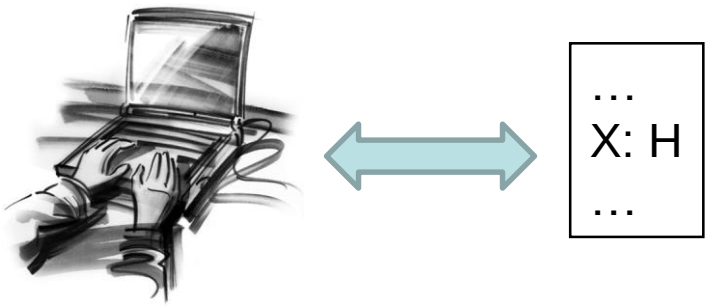
Without salt:

1. User types userid, X, and password, P.

2. System looks up H, the stored hash of X's password.

3. System tests whether h(P) = H.

Password file:



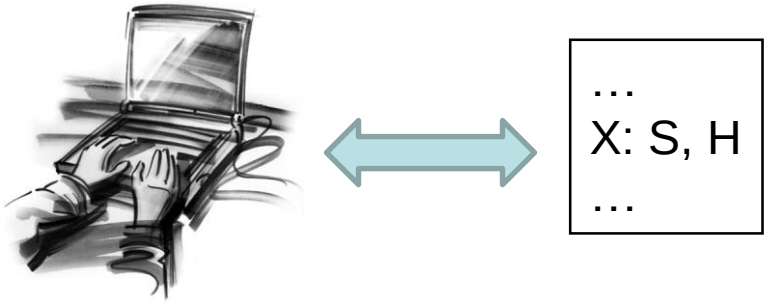
With salt:

1. User types userid, X, and password, P.

2. System looks up S and H, where S is the random salt for userid X and H is stored hash of S and X's password.

3. System tests whether h(S||P) = H.

Password file:



How Salt Increases Search Space Size

- Requires re-computation of the dictionary for each attack attempt of a new system
- For a salt of B bits, a pre-computed dictionary attack requires 2^B times larger storage (and preparation time)

How Salt Increases Search Space Size

- Assuming that an attacker cannot find the salt associated with a userid he is trying to compromise, then the search space for a dictionary attack on a salted password is of size

  $$2^B * D,$$

  where B is the number of bits of the random salt and D is the size of the list of words for the dictionary attack.
- For example, if a system uses a 32-bit salt for each userid and its users pick passwords in a 500,000 word dictionary, then the search space for attacking salted passwords would be

  $$2^{32} * 500{,}000 = 2{,}147{,}483{,}648{,}000{,}000,$$

  which is over 2 quadrillion.
- Also, even if an attacker can find a salted password for a userid, he only learns that one password.

Password salting

- Does not improve security of a single password, but makes pre-computed dictionary attacks against a large set of passwords less effective
  - Salting is effective against both pre-computed dictionary attacks and rainbow tables

- A password guess cannot be tried for all users simultaneously. Why?
  - Two users with the same password have different entries in the password file

Password Authentication in Windows

- Password hashes are stored in a file called *Security Account Manager (SAM)* file, which is not accessible to regular users
  - Passwords are hashed using a combination of MD4 hash function and a custom-brewed LAN Manager (LM) hash function

Password Authentication in Linux

- Uses a random salt of at least 12 bits
- /etc/passwd file
  - Contains username, userid, full name, path, etc.
  - It is world-readable
- /etc/shadow file
  - Contains salt, hashed password
  - It is readable only by super-user

Password Authentication in Linux

- Passwords are stored in /etc/passwd file, usually in conjunction with /etc/shadow file
- Format of shadow file

vivek:1fnfffc$pGteyHdicpGOfffXX4ow#5:13064:0:180:7:::

  - User name
  - Hash value: $algo$salt$hash
    - Algorithm can be 1 (md5), 2 (blowfish), 5 (sha256), 6 (sha512)
  - Last password change (number of days since Jan 1, 1970)
  - Minimum number of days before password may be changed
  - Maximum number of days before which password must be changed (99999 is indefinite).
  - Number of days before password expiration when user will be warned
  - Other fields: Number of days after password expiration when account will be disabled; Number of days since January 1, 1970 that an account has been disabled; Field is reserved for future use.

One-time passwords

- Fixed passwords are vulnerable to eavesdropping and replay attacks
- One-time password: each password is only used once!
- Initialization (executed over secure channel):
  - User has secret pswd
  - User uses one-way function h to compute anchor A:

    $A = h^n(pswd) = h(h(\ldots(h(pswd))\ldots)$; (h applied n times)
  - User sends length L = n and anchor $A = h^n(pswd)$ to server for storage
- Authentication in the i-th session
  - Server sends current length L to client
  - User uses password $P = h^{L-1}(pswd)$
  - Server checks if $h^{n+1-L}(P)$ equals anchor A
  - Server decrements length: L = L-1

One-time passwords

- Server has L = n and anchor A = h^n(pswd)
- Client knows pswd
- To authenticate:
  - 1st authentication:
    - Client ⇒ Server: "I want to authenticate"
    - Server ⇒ Client: L = n
    - Client ⇒ Server: P = h^{n-1}(pswd)
    - Server computes h^1(P) and compares with h^n(pswd); they are equal!
    - Server updates L = L-1 (L now equals n-1)
  - 2nd authentication
    - Client ⇒ Server: "I want to authenticate"
    - Server ⇒ Client: L = n-1
    - Client ⇒ Server: P = h^{n-2}(pswd)
    - Server computes h^2(P) and compares with h^n(pswd); they are equal!
    - Server updates L = L-1 (L now equals n-2)

One-time passwords

- Essentially, the password h^{n-i}(pswd) is used <u>one time</u> for authentication in session i

- An attacker that learns the password for session i , h^{n-i}(pswd), cannot derive the password for the next session, session i+1, which is $h^{n-(i+1)}$(pswd)
  - Because of the one-way property of the hash function

Real world attacks against passwords

- In June 2012, the online dating site eHarmony suffered a data breach
  - More than 1.5 million password hashes were stolen and later dumped online by a hacker gang called Doomsday Preppers
  - It took security company Trustwave SpiderLabs only 72 hours to crack about 80% of these password hashes

More on the eHarmony password cracking

http://blog.spiderlabs.com/2012/06/eharmony-password-dump-analysis.html

- Password cracking was performed on a custom built system using off-the-shelf parts totaling less than $1,500 utilizing three NVIDIA 460GTX graphics cards (GPUs), using some standard cracking tools like Hashcat and John the Ripper
- Passwords were stored in a non-salted MD5 format
- Passwords were very easy to crack mostly because they were not salted and because they were stored case insensitive (which reduced considerably the possible password space)
- Many of the passwords used by eHarmony users were names of sports teams, dogs, states, and masculine and feminine names

More on the eHarmony password cracking

http://blog.spiderlabs.com/2012/06/eharmony-password-dump-analysis.html

Conclusion:

"The eHarmony dump is just further proof that organizations need to not only store passwords in stronger, salted formats than was previously acceptable, but also need to enforce stronger case-sensitive password policies. Users, as a whole, still do not understand the need for strong passwords, and will continue to set passwords that meet only the minimum requirements."

LinkedIn password hack

- In June 2012, a file containing 6.5 million unique hashed passwords appeared in an online forum based in Russia
  - More than 60% of these passwords have reportedly been cracked so far
- The file only contains passwords hashed using the SHA-1 algorithm and does not include user names or any other data
  - <u>Passwords were not salted</u>
- It is likely that this file contains LinkedIn passwords because many of the cracked passwords contain the word “LinkedIn”
  - Numerous anecdotal reports that users have seen their LinkedIn password posted online

LinkedIn password hack

- On June 6, 2012, an official LinkedIn blog post confirms that "some of the passwords that were compromised correspond to LinkedIn accounts."

http://blog.linkedin.com/2012/06/06/linkedin-member-passwords-compromised/

- The same blog post mentions that LinkedIn has switched to using a salted password file:
  - "the affected members who update their passwords and members whose passwords have not been compromised benefit from the enhanced security we just recently put in place, which includes hashing and salting of our current password databases."

LinkedIn password hack

- How fast can unsalted passwords be cracked?

To get an idea, from this blog post:

http://erratasec.blogspot.com/2012/06/confirmed-linkedin-6mil-password-dump.html

“The answer is "2 billion per second" using the Radeon HD 7970 (the latest top-of-the-line graphics processor). Each letter of a password has 100 combinations (UPPER, lower, d1g1ts, $ymbols). A 5 letter password therefore has 100 x 100 x 100 x 100 x 100 or 10 billion combinations, meaning it can be cracked in 5 seconds. A 6 letter password has 100 times that, or 500 seconds. A 7 letter password has 100 times that, or 50,000 seconds, or 13 hours. An 8 character password is roughly 57 days. A 9 character password is 100 times that, about 15 years. In other words, if your password was 7 letters, the hacker has already cracked it, but if it's 9 letters, it's too difficult to crack with brute force. “

How fast can passwords be cracked?

- A cluster of five, 4U servers equipped with 25 AMD Radeon GPUs and communicating at 10 Gbps over Infiniband switched fabric was able to churn through 348 billion NTLM password hashes per second

- https://securityledger.com/2012/12/new-25-gpu-monster-devours-passwords-in-seconds/
- http://www.hpcwire.com/2012/12/06/gpu_monster_shreds_password_hashes/

Unsalted passwords are useless

- Rainbow tables are dead:

http://blog.ircmaxell.com/2011/08/rainbow-table-is-dead.html

- A list of websites that are known/suspected to store passwords in plaintext:

  http://plaintextoffenders.com/

Other recent password stealing incidents

- June 2011: over 1 million SonyPictures.com passwords compromised
- February 2013: hack attack exposes password data for 250,000 Twitter users
- April 2013: 50 million LivingSocial passwords were stolen
- February-March 2014: eBay database that contained passwords was breached

Password Managers

- A password manager is a software application that helps a user store and organize passwords
- The user needs to remember only one master password (supposedly a strong password)
  - All the other passwords are usually stored encrypted with the master password
  - Read more about password managers at:

http://en.wikipedia.org/wiki/Password_manager

Password Managers

- A research article at a recent conference (Aug 2014) studied several online password managers and found vulnerabilities
  - LastPass, RoboForm, My1login, PasswordBox, NeedMyPassword

http://www.csoonline.com/article/2453941/identity-access/why-password-managers-are-not-as-sec/identity-access/why-password-managers-are-not-as-secure-as-you-think.html

- Full article available at:

https://devd.me/papers/pwdmgr-usenix14.pdf

"The root causes of the vulnerabilities are also diverse: ranging from logic and authorization mistakes to misunderstandings about the web security model... Our study suggests that it remains to be a challenge for the password managers to be secure."

Password Managers

- Subsequent issues with LastPass:
  - 2016:

    https://www.bankinfosecurity.com/lastpass-patches-password-manager-vulnerability-a-9299

  - 2019:

    https://arstechnica.com/information-technology/2019/09/lastpass-fixes-bug-that-leaked-the-password-of-last-logged-in-account/

Graphical passwords

Graphical passwords

- Why? Text-based passwords are predictable and vulnerable to dictionary attacks
- Motivation for graphical passwords
  - They do not need any additional hardware
  - Humans tend to remember pictures better than text
  - Better resistance to dictionary attacks
  - Greater password space with large number of possible pictures

Motivation for graphical passwords

GrAphICAl

LacLHPaRg

g18aP9c1L

Which one is easier to remember?

Graphical Passwords – 2 categories

- Two distinct categories
  - Recognition-based schemes
    - Image Select – Dhamija and Perrig
    - Shoulder Surfing – Sobrado and Birget
    - Face Scheme
    - Story Scheme
    - User study of Face & Story Scheme
  - Recall-based schemes

Image Selection – Dhamija and Perrig

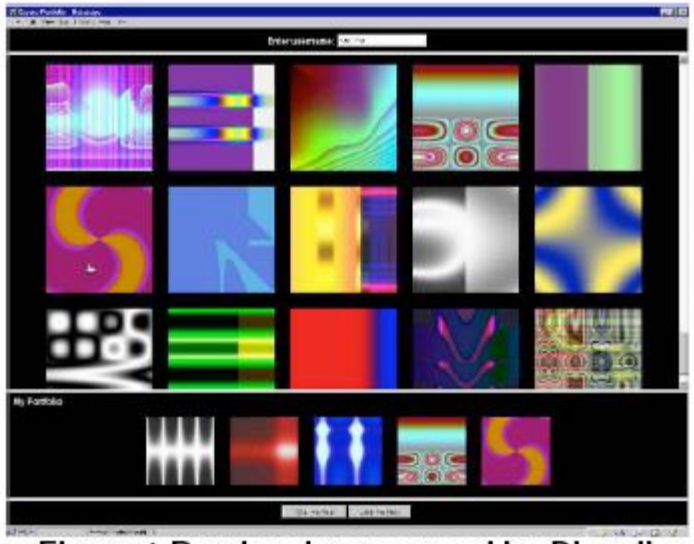


Figure 1.Random images used by Dhamija and Perrig [4]

- Users asked to select several randomly generated images from a large set
- User must identify his/her selected images to authenticate (in order!)
- User study revealed 90% authentication success
- Log-in time is long
- Server must store image seeds (random seeds)

Shoulder Surfing resistant – Sobrado and Birget

Figure 2. A shoulder-surfing resistant graphical password scheme [12]

Sobrado and Birget suggested using 1000 objects for each display!

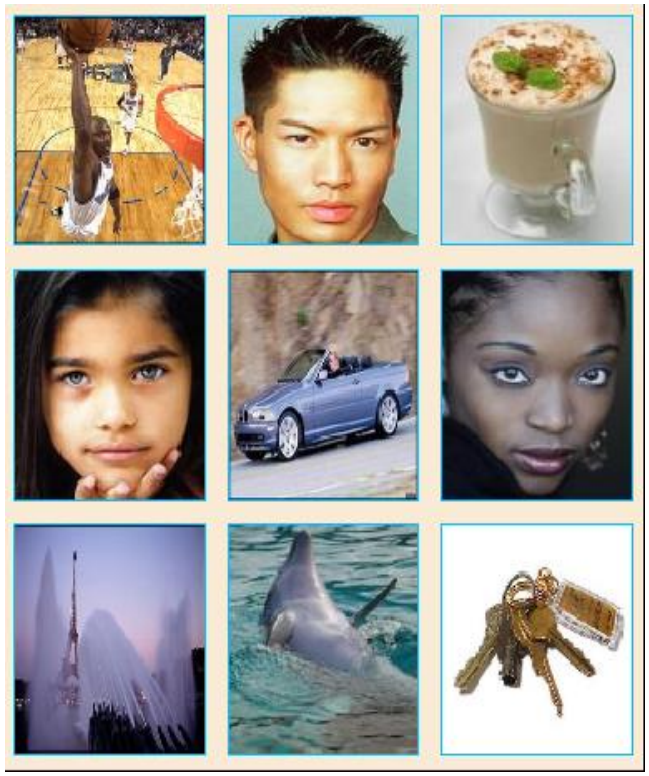
- User pre-selects a number of objects
- User must identify those objects among others
- To authenticate, must click inside the shape formed by the pass-objects

Face Scheme

A password is a group of faces

- A password is a collection of *k* faces
- Each face is selected from a screen with *n* faces
- This yields n^k possible passwords
- To authenticate, the same *k* screens are shown, but with the faces randomly permuted

Story Scheme



A password is a sequence of images

- Password is a sequence of k images from a set of n images (with $n>k$)
- This yields $n!/(n-k)!$ passwords
- Nine categories of images
  - Drawn from a variety of sources
  - Carefully balanced
- To authenticate, user must select images in sequence order on a screen with same images (but shown permuted)

User Study

- Total of 154 university subjects
- Randomly assigned to either Face of Story
- Graphical password used to gain access to course content
- No requirement to change passwords

Population		Scheme	
Gender	Race	Face	Story
any	*any*	79	95
Male	*any*	55	77
Female	*any*	20	13
Male	Asian	24	27
Female	Asian	12	8
Male	Black	3	-
Female	Black	-	-
Male	Hispanic	-	2
Female	Hispanic	-	-
Male	White	27	48
Female	White	8	4

Population breakdown (in passwords)

User Study

- 154 users chose 174 passwords
- 2271 login attempts out of 2648 were successful (85.76%)
- At the conclusion of the study, each user completed an exit survey.

> "To start, I chose a face that stood out from the group, and then I picked the closest face that seemed to match."

Guessing Entropy

- "Guessing Entropy measures the expected number of guesses an attacker with perfect knowledge of the probability distribution on passwords would need in order to guess a password chosen from that distribution"
- G_s^{avg} is average over all passwords
- G_s^{med} is median over all passwords
- G_s^{25} is number of guesses to find 25% of passwords
- G_s^{10} is number of guesses to find 10% of passwords

User Study Results

Population	$G_{\mathcal{S}}^{\text{avg}}$	$G_{\mathcal{S}}^{\text{med}}$	$G_{\mathcal{S}}^{25}$	$G_{\mathcal{S}}^{10}$
Overall	790	428	112	35
Male	826	404	87	53
Female	989	723	125	98
White Male	844	394	146	76
Asian Male	877	589	155	20

Table 2: Results for Story, $\lambda_0 = 2^{-2}$

- Note that for Story, there are 3024 possible passwords with $n = 9$ and $k = 4$
- This means max G is 1513
- Higher G^{avg} than G^{med} indicates a few good password choices and many poor choices

- For Face, there are 6561 possible passwords with $k = 4$
- Max G is 3281
- Higher G^{avg} than G^{med} indicates a few good password choices and many poor choices

Population	$G_{\mathcal{S}}^{\text{avg}}$	$G_{\mathcal{S}}^{\text{med}}$	$G_{\mathcal{S}}^{25}$	$G_{\mathcal{S}}^{10}$
Overall	1374	469	13	2
Male	1234	218	8	2
Female	2051	1454	255	12
Asian Male	1084	257	21	5.5
Asian Female	973	445	19	5.2
White Male	1260	81	8	1.6

Table 3: Results for Face, $\lambda_0 = 2^{-2}, \lambda_1 = 2^2$

User Study Results

Pop.	Female Model	Male Model	Typical Female	Typical Male
Female	40.0%	20.0%	28.8%	11.3%
Male	63.2%	10.0%	12.7%	14.0%

Table 4: Gender and attractiveness selection in Face.

- Female faces were the most popular choice for either gender

"I simply picked the best lookin girl on each page."

- Asian Females and White Females picked within their race 50% of the time
- White Males; 62% of the time chose within their race
- Statistics on Black Males based on only 3 study participants; needs additional validation

Pop.	Asian	Black	White
Asian Female	52.1%	16.7%	31.3%
Asian Male	34.4%	21.9%	43.8%
Black Male	8.3%	91.7%	0.0%
White Female	18.8%	31.3%	50.0%
White Male	17.6%	20.4%	62.0%

Table 5: Race selection in Face.

Recall Based Techniques

- Reproduce a Drawing
  - Jermyn, et al. – Draw – a – secret (DAS)
- Repeat a Sequence
  - Passlogix (based on Blonder)
  - Wiedenbeck, et al. – Passpoint
  - v–Go by Passlogix

DAS - Jermyn, et al.

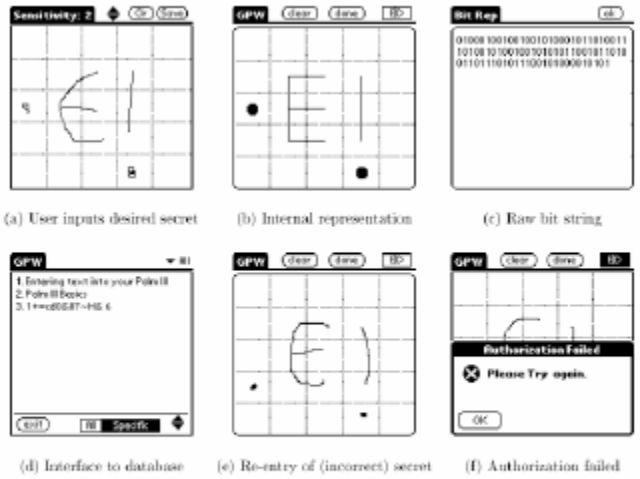


Figure 6. Draw-a-Secret (DAS) technique proposed by Jermyn, et al. [24]

- Passwords are a simple image drawn on a grid
- During authentication, user must draw password again
  - If the drawing touches the same grids in the same sequence, the user is authenticated

Passlogix based on Blonder

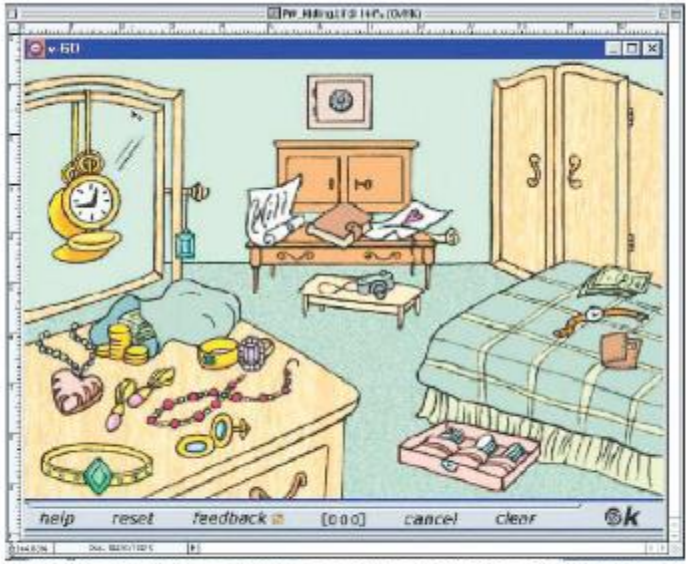


Figure 9. A recall-based technique developed by Passlogix [34]

- Various items in the scene are clicked on to form a password
- To authenticate, users must click the same items in the same order
- Requires that each scene be divided into items with click-boundaries

Passpoint – Wiedenbeck, et al.

Figure 10. An image used in the Passpoint Sytem, Wiedenbeck, et al. [35]

- Unlike Passlogix, allows arbitrary images
- Anywhere on the image can be a password point
- A tolerance around the selected pixels is calculated
- To authenticate, users must pick the same pixels within that tolerance

v-Go by Passlogix

- Users mix a virtual “cocktail” and its ingredients become their password
- Many options are available
  - Picking a hand at cards
  - Preparing a “meal” in a virtual kitchen
- No good way to prevent poor password choice
  - E.g., a full house in hand of cards
- Limited password spaces

Graphical passwords - Conclusion

- Is a graphical password as secure as text-based password?
  - Brute force search
    - Password spaces
    - Difference in difficulty
  - Dictionary attacks
    - Mouse vs. keyboard entry
  - Guessing
    - Weak password choices
    - Predictability

Graphical passwords - Conclusion

- Is a graphical password as secure as text-based password?
  - Spyware
    - Mouse vs. keyboard
  - Shoulder Surfing
    - Some graphical passwords schemes provide protection
    - Shared problem
    - Recall-based schemes are vulnerable
  - Social Engineering
    - Harder to share graphical passwords

Graphical passwords - Conclusion

- Growing interest over the past decade in Graphical Passwords
- Main argument is they are easier to remember
  - Graphical passwords need fewer attempts to authenticate, but take longer to learn and take longer to input during authentication.
  - Questions remain about the validity of this argument
- Preliminary analysis suggests increased resistance to traditional attacks
- Lacking wide-scale deployments, difficult to fully analyze vulnerabilities
- Graphical Password systems are immature

Graphical passwords - Conclusion

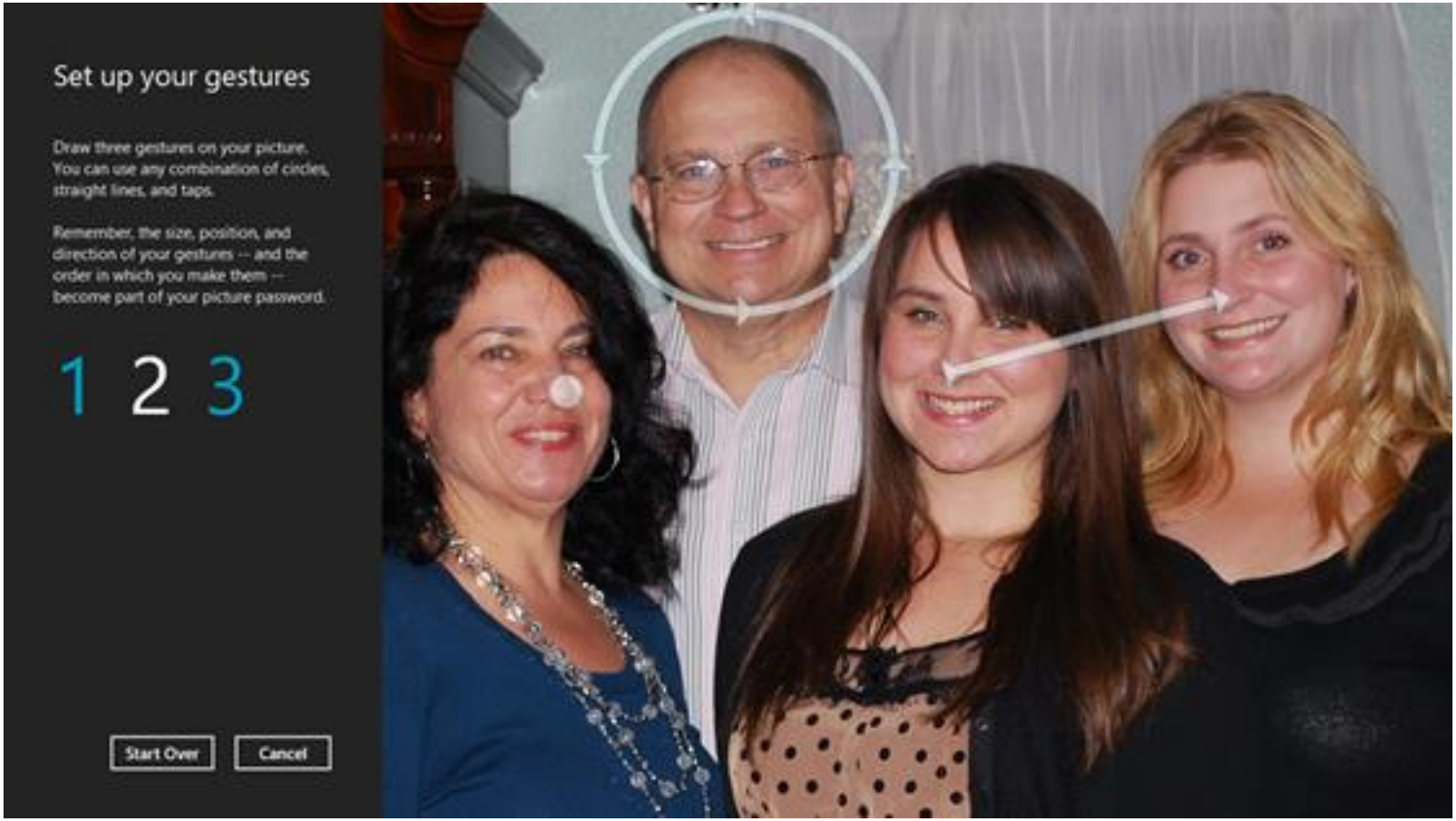
- For “Face” scheme, user choice is so biased, that it renders the scheme insecure
- How to mitigate this threat?
  - Limit user choice
  - Educate users to choose passwords better
  - Select images that are less prone to these types of biases (such as in the “Story” scheme)

Windows 8 (and 10) – Picture Password

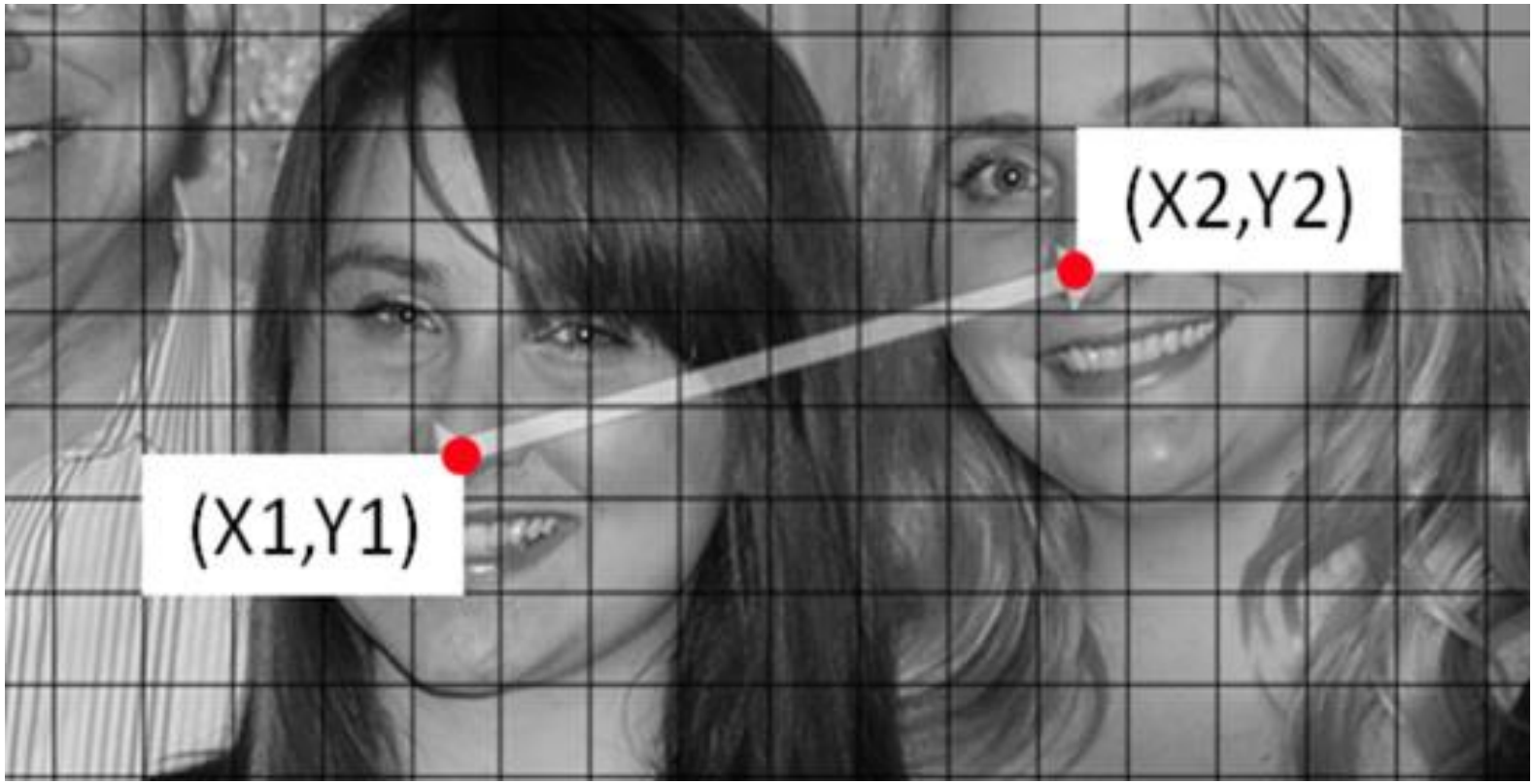
- Windows 8 has a “picture password” option
- User selects the picture (this could improve security and memorability of the password)
- User defines gestures on the picture (defines areas and ways to connect those areas) through circles, straight lines and taps.
- User confirms gestures

- Works both for touch screen devices (tablet) and PCs using the mouse
- After 5 unsuccessful attempts, user is locked out and asked to enter textual password

Windows 8 – Picture Password



How does it work?



Security of the scheme?

- Shoulder surfing attack
- The smudge attack: people can see the traces of fingers on the touch screen
- Other potential attack: user predictability (users choose a predictable set of locations/gestures)
- Weakest link attack: password security is as good as the weaker between the picture and textual password (because of the lock-out mechanism)

Smudge attacks?

Security of the scheme?

- More about the security of this scheme and an analysis of the password space size:

http://blogs.msdn.com/b/b8/archive/2011/12/16/signing-in-with-a-picture-password.aspx

- A follow-up post by Microsoft researchers with advice to users on how to choose a hard-to-guess picture password:

https://blogs.msdn.com/b/b8/archive/2011/12/19/optimizing-picture-password-security.aspx

- More follow-up articles:
  - http://www.networkworld.com/news/2013/090913-windows8-273634.html
  - http://security.stackexchange.com/questions/20228/how-secure-is-windows-8s-picture-password-login

What to read?

- Chapters 1.4.2, 3.3.2 from the Textbook
- Posted article on the course website:
  - J. Bonneau, C. Herley, P. C. Van Oorschot, and F. Stajano, "Passwords and the Evolution of Imperfect Authentication", *In Communications of the ACM*, 2015
- For graphical passwords:
  - Darren Davis, Fabian Monrose, and Michael K. Reiter, "On user choice in Graphical Password Schemes", *In Proceedings of the 13th USENIX Security Symposium*, August, San Diego, 2004
  - Xiaoyuan Suo, Ying Zhu, and G. Scott Owen, "Graphical Passwords: A Survey", *In Proceedings of the 21st IEEE Annual Computer Security Applications Conference*, 2005.